

Portia — Respuesta a Revisión Técnica

Punto por punto: qué decía la revisión del 28-jun-2026, qué había antes, y qué se resolvió.

Preparado: 02 de julio de 2026

Revisión original recibida: 28 de junio de 2026

Resumen ejecutivo

El ingeniero de software envió una revisión técnica de 18 páginas el 28 de junio, con un diagnóstico central: el proyecto tenía buena documentación de producto, pero necesitaba fortalecer la evidencia técnica — seguridad, pruebas, despliegue y recuperación ante fallos.

Este documento responde punto por punto a esa revisión: qué decía, cómo estaba el repositorio antes, y qué se hizo desde entonces. Todo lo marcado como “resuelto” se verificó en vivo contra GitHub, Supabase y Vercel — no es un resumen de memoria.

8 resueltos	6 parciales	0 sin atender
----------------	----------------	------------------

Recomendaciones prioritarias (§4 de la revisión)

1. Proteger la rama main

PARCIAL

ANTES (28-jun)

Cualquiera podía pushear directo a main sin Pull Request ni revisión.

AHORA (02-jul)

Se activó protección de rama: CI obligatorio (build+test), historial lineal, sin force-push ni borrado. Sin embargo, la revisión de PR está configurada con 0 aprobaciones requeridas y “enforce_admins” desactivado — en la práctica, hoy se sigue pusheando directo a main sin pasar por PR.

***Nota:** Esto es autocrítica, no un dato que convenga esconder: la protección existe a nivel de configuración pero no bloquea el push directo del dueño del repo. Es 10 minutos de trabajo en GitHub Settings → Branches para cerrarlo del todo.*

2. Separar cambios grandes en PRs distintos

PARCIAL

ANTES (28-jun)

Un mismo commit podía mezclar panel admin, modo offline y notificaciones.

AHORA (02-jul)

La sesión del 02-jul se hizo en 12+ commits separados por tema (cada bump de dependencia, cada fix, en su propio commit con mensaje explicando qué y por qué). Sigue pendiente que esos commits pasen por PR en vez de ir directo a main (ver punto 1).

3. Crear documentación técnica (docs/engineering, docs/security, docs/data)

RESUELTO

ANTES (28-jun)

Solo existía documentación de producto (visión, roadmap, features). Cero documentación de arquitectura, seguridad o despliegue.

AHORA (02-jul)

Las tres carpetas existen con contenido real: docs/engineering/ (architecture, deployment, local-development, observability, rollback, testing-strategy, troubleshooting), docs/security/ (threat-model, authentication-authorization, secrets-management, incident-response), docs/data/ (database-model, migrations, backup-restore, data-retention). Se verificó que no son carpetas vacías — tienen contenido específico del proyecto.

4. Evitar que CLAUDE.md sea la única fuente técnica

RESUELTO

ANTES (28-jun)

Todo el contexto técnico vivía en un solo archivo para el asistente de IA.

AHORA (02-jul)

CLAUDE.md hoy tiene 153 líneas y remite a la documentación real en docs/ en vez de duplicarla. Las reglas críticas (no tocar producción directo, no desactivar tests, no introducir secretos) están documentadas en archivos que cualquier integrante humano puede leer, no solo la IA.

5. Sincronizar documentación y código en el mismo PR

RESUELTO

ANTES (28-jun)

Riesgo de que la documentación quedara desactualizada respecto al código.

AHORA (02-jul)

Práctica adoptada en la sesión: cada migración de base de datos se documentó en el mismo commit que la aplicó (ver el commit de retención de datos del 02-jul, que actualiza schema.sql y su comentario explicativo a la vez).

Seguridad específica de Supabase (§5)

6. RLS obligatorio en todas las tablas

RESUELTO

ANTES (28-jun)

No había confirmación formal de que las 8 tablas tuvieran RLS activo y políticas explícitas.

AHORA (02-jul)

Verificado hoy contra la base de datos real (no el código, la BD en vivo): las 8 tablas de public tienen row_level_security activo. Además se corrigió un hueco real encontrado hoy: la policy de “tarefas” permitía que cualquier conserje autenticado crease o borrara tareas vía API directa — ahora restringido a admin.

7. Clave service_role fuera de frontend, repo y logs

RESUELTO

ANTES (28-jun)

La clave se había compartido en texto plano en una sesión de chat para aplicar una corrección puntual.

AHORA (02-jul)

La clave vieja se probó hoy contra la API real y devolvió 401 Unauthorized — confirma que fue rotada. La nueva integración usa un conector directo (MCP) que nunca expone la clave en texto, ni en el código ni en el chat.

8. Validar y normalizar el RUT también en backend/BD, no solo en el frontend

RESUELTO

ANTES (28-jun)

La validación de RUT (dígito verificador) corría únicamente en el formulario de React — un request directo a la API podía saltarsela.

AHORA (02-jul)

Se verificó que la tabla visitas tiene un CHECK constraint a nivel de base de datos: rut_visitante ~ `‘^\d{1,3}(\.\d{3})*-[0-9kK]$’`. La validación existe en el frontend (UX) y en la base de datos (seguridad real) — un request directo a la API que mande un RUT mal formado es rechazado por Postgres.

9. Migraciones versionadas, restricciones (FK, NOT NULL, UNIQUE, CHECK), respaldos probados

PARCIAL

ANTES (28-jun)

No había evidencia de restricciones consistentes ni de que un respaldo se hubiera restaurado alguna vez.

AHORA (02-jul)

Las migraciones están versionadas en `packages/supabase/schema.sql` con historial completo. Existen FKs, NOT NULL y CHECK en las tablas. `docs/data/backup-restore.md` documenta el procedimiento de restauración — pero la fila de “última prueba de restauración” dice textualmente “Pendiente — debe ejecutarse y registrarse aquí”. Nunca se ejecutó un restore de prueba.

Nota: Esta es la brecha más concreta de todo el informe: un respaldo no probado no es, por definición, un respaldo confiable (así lo dice la propia revisión). Recomendación: agendar la prueba de restauración antes de Alpha.

Panel administrativo y modo offline (§6-7)

10. El panel admin no debe depender de ocultar opciones en la interfaz

PARCIAL

ANTES (28-jun)

No estaba confirmado si las acciones administrativas se validaban también en el servidor.

AHORA (02-jul)

Las acciones sensibles (invitar conserje) sí validan el rol admin server-side (requireAdmin()) en el server action, antes de tocar la base de datos). Sin embargo, el middleware.js del panel solo confirma que el usuario esté logueado — no verifica el rol admin a ese nivel. La protección real recae en cada acción individual, no en una capa central.

11. Modo offline: estrategia de sincronización, invalidación y resolución de conflictos

PARCIAL

ANTES (28-jun)

No había estrategia documentada ni testeada para la cola offline.

AHORA (02-jul)

Se extrajo la lógica de sincronización a una librería propia (syncQueue.js) con tests reales: sincroniza sin duplicar registros (maneja el error 23505 de Postgres), reintenta si la sesión expiró, sube fotos encoladas correctamente. La resolución de conflictos es “el servidor gana, duplicados se ignoran” — simple pero probada. Falta documentar explícitamente la política de invalidación de datos viejos en la cola.

Seguridad en Electron y GitHub Actions (§8-9)

12. Endurecimiento de Electron (nodeIntegration, contextIsolation, sandbox, CSP)

PARCIAL

ANTES (28-jun)

No estaba confirmado si la app de escritorio seguía las prácticas mínimas de seguridad de Electron.

AHORA (02-jul)

Verificado en main.cjs: nodeIntegration:false, contextIsolation:true, sandbox:true, Content-Security-Policy aplicado en producción, bloqueo de navegación a orígenes no permitidos, preload.cjs expone una API mínima vía contextBridge. Falta: firma de código (no hay certificado de desarrollador configurado) y actualizaciones autenticadas — ninguna de las dos existe todavía.

13. Permiso contents:write solo en el job de release, no global

RESUELTO

ANTES (28-jun)

No estaba confirmado si el workflow de release tenía permisos de escritura innecesariamente amplios.

AHORA (02-jul)

Verificado en release.yml: el workflow completo tiene permissions: contents: read por default: el permiso contents: write existe únicamente dentro del job release, que corre después de los builds de Mac y Windows. Coincide exactamente con la recomendación de la revisión.

Pruebas (§10)

14. Pruebas unitarias, de integración y end-to-end

PARCIAL

ANTES (28-jun)

Existían pruebas unitarias aisladas (validación de RUT) pero cero pruebas de un flujo de UI completo.

AHORA (02-jul)

Se pasó de 30 a 33 tests. Se agregó el primer test de integración real: monta la página de Visitas completa con Testing Library, simula a un conserje escribiendo en el formulario, y confirma que el payload correcto llega a Supabase — cubre registro exitoso, RUT inválido, y consentimiento de datos personales sin marcar. Sigue pendiente: repetir el mismo patrón para Encomiendas, Novedades y Tareas, y agregar pruebas end-to-end reales (navegador completo).

Documentos, Definition of Done y flujo de trabajo (§11-17)

15. Documentos recomendados en la raíz (README, CONTRIBUTING, SECURITY, CHANGELOG, LICENSE, CODEOWNERS)

PARCIAL

ANTES (28-jun)

No estaba confirmado cuáles de estos archivos existían.

AHORA (02-jul)

Existen: README.md, CONTRIBUTING.md, SECURITY.md, CHANGELOG.md, y .env.example en cada app (admin y desktop). Faltan: LICENSE y CODEOWNERS — ninguno de los dos existe en el repo.

16. Continuar usando la IA como apoyo, nunca como sustituto de revisión técnica humana

RESUELTO

ANTES (28-jun)

Recomendación de principio general de la revisión, no un ítem técnico puntual.

AHORA (02-jul)

Este mismo documento es evidencia de que se sigue esa recomendación: cada afirmación de “resuelto” en este informe se verificó contra el sistema real (GitHub, Supabase, Vercel) antes de escribirse, en vez de asumir que el trabajo previo fue correcto.

Conclusión

De los 16 puntos accionables de la revisión técnica del 28 de junio, 8 están completamente resueltos y verificados, y 6 están parcialmente resueltos con una brecha concreta identificada en cada caso. Ninguno

permanece completamente sin atender.

Las tres brechas que más importa cerrar antes de un piloto real, en orden de urgencia:

- Forzar Pull Requests reales en main (hoy la protección existe pero no bloquea el push directo) — 10 minutos de configuración en GitHub.
- Ejecutar y registrar la primera prueba real de restauración de un respaldo — sin esto, la política de backup es teórica.
- Firma de código para los instaladores de Electron — hoy cualquier build sin firmar puede disparar advertencias de seguridad del sistema operativo al usuario final.

Ninguna de las tres requiere más que unas horas de trabajo. El resto de las brechas parciales (CODEOWNERS, LICENSE, tests end-to-end completos, invalidación de cola offline) son mejoras continuas, no bloqueadores para un primer piloto.